JOIN PIPELINE OPTIMIZATION

COMPLETE TECHNICAL REPORT

Stefanos Tzaferis
Eleftheria Galiatsatou

Experimental Setup

All experiments were conducted on the university's Linux systems. We aimed to run tests during low-load periods to ensure consistent performance. Execution times were averaged over multiple runs (typically 4-5) to stabilize results.

CPU	Intel(R) Core(TM) i5-6500 @ 3.20GHz (4 Cores)
Operating System	Ubuntu 20.04.6 LTS (Focal Fossa)
Compiler	gcc 9.4.0
Profiling	getrusage for Memory, chrono for Timing

Phase 1: Custom Hash Join Implementations

During this phase, we implemented three distinct hashing algorithms to replace the baseline std::unordered_map. While we achieved significant speedups over the course of the project, the standalone hash joins showed varying results compared to the baseline.

1.1 Baseline Performance

Average Baseline Time: 276,637 ms.

1.2 Robin Hood Hashing

Average Time: 271,414 ms. This algorithm performed slightly better than the baseline. We implemented it as a multimap where duplicate keys are handled via a vector stored within each hash node. This encapsulated approach provided better performance and cleaner code than managing duplicates externally.

Key Optimizations:
• Hash Function: Switching from std::hash to MurmurHash3 finalizer significantly improved performance.
• Initial Capacity: Set to double the size of the build table to prevent expensive rehashing.
• Load Factor: Experimentation showed 0.75 was optimal.
• Overheads: Small getters/setters were inlined, and std::move was used extensively to avoid copying large objects.

1.3 Hopscotch Hashing

Average Time: 309,102 ms. We were unable to outperform the baseline with this implementation. We maintained std::hash as others did not yield better results here.

Key Optimizations:
• Bitmap Efficiency: The H-sized neighborhood bitmap vector was replaced with an H-bit integer. This allowed for bitwise operations and avoided dynamic memory allocations during neighborhood searches.
• Configuration: Neighborhood size (H) was set to 32 bits, balancing bitmap size with insertion success rates.
• Rehashing: Triggered if the neighborhood is full, if no free slot is found, or if swapping fails. During rehash, duplicate vectors were moved using std::move to preserve efficiency.

1.4 Cuckoo Hashing

Average Time: 289,445 ms. Implemented as a multimap with vectors for duplicates inside the hash table.

Key Optimizations:
• Capacity Logic: The initial capacity applies to both sub-tables. The load factor is total size / (2 * capacity), which helps delay rehashing even if one table fills up due to hash distribution.
• Hashing: Used a combination of MurmurHash3 and split64 with different seeds.
• Bitwise Operations: Capacities are powers of 2, allowing bitwise AND instead of modulo for boundary checks, saving ~15 seconds.
• Logic Refinement: Removed the "already exists" check inside the kick loop, as kicked values cannot exist in multiple buckets.Phase 2: Structural Optimizations

2.1 Late Materialization

Execution Time: 182,186 ms (-34% from base). Peak memory usage was reduced by nearly 50%.

Technical Specifications:
• Type-Tagging: Developed a custom value_t using bit-masking in the dataIdx field. Three Most Significant Bits (MSB) are used for tagging types: NULL (000), INT32 (010), regular VARCHAR (100), and long VARCHAR (101).
• Masks: TYPE_MASK (0xE000) for tag isolation and OFFSET_MASK (0x1FFF) for actual data indexing within a page.
• Performance: Avoided the performance penalty of std::variant for large datasets.

2.2 Column-Store Table (ColumnT)

Execution Time: 106,616 ms (-61% from base).

Implementation Details:
• Layout: Intermediate results are stored in a paged columnar format. Each column (ColumnT) includes a pageRowOffset vector.
• Data Access: getValueAtRow(rowIdx) utilizes std::upper_bound (Binary Search) on pageRowOffset to find the correct page in O(log n) time, drastically speeding up access to intermediate result rows.

2.3 Unchained Hash Table

Execution Time: 62,034 ms (-77.5% from base).

Implementation Details:
• Adjacency Array: Tuples are stored in a contiguous buffer sorted by hash prefixes, eliminating pointer chasing.
• Directory Slots: Directory entries use 48 bits for pointers and 16 bits for Bloom filters. CRC32 hardware-accelerated hashing was used.
• Logic: Search logic utilizes Bloom filters to skip non-matching buckets, followed by a linear scan within the adjacency range.

Phase 3: High-Performance Optimizations

3.1 Zero-Copy Indexing Optimization

Execution Time: 39,256 ms (-85% from base).

Implementation Details:
• Density Detection: The algorithm parses page headers (first two uint16_t fields) to check if a column is "dense" (INT32 with no NULLs).
• Zero-Copy: If a column is dense, the ColumnT is marked as copied = false. Instead of duplicating data, it holds a colRef to the original input Column. Data is accessed directly from the input pages using reinterpret_cast, significantly reducing CPU and memory overhead.

3.2 Parallel Build Phase & Slab Allocator

Implementation Details:
• Three-Level Slab Allocator: Designed to avoid global memory allocator contention. Levels include Global (Large Chunks), Small Chunks (per partition), and Individual Tuples. This reduced system calls during heavy data materialization.
• Build Logic: Threads collect tuples into local collectors/partitions. A single thread per partition then merges linked lists of chunks to form the final sorted adjacency array.

3.3 Parallel Probe Phase & Work Stealing

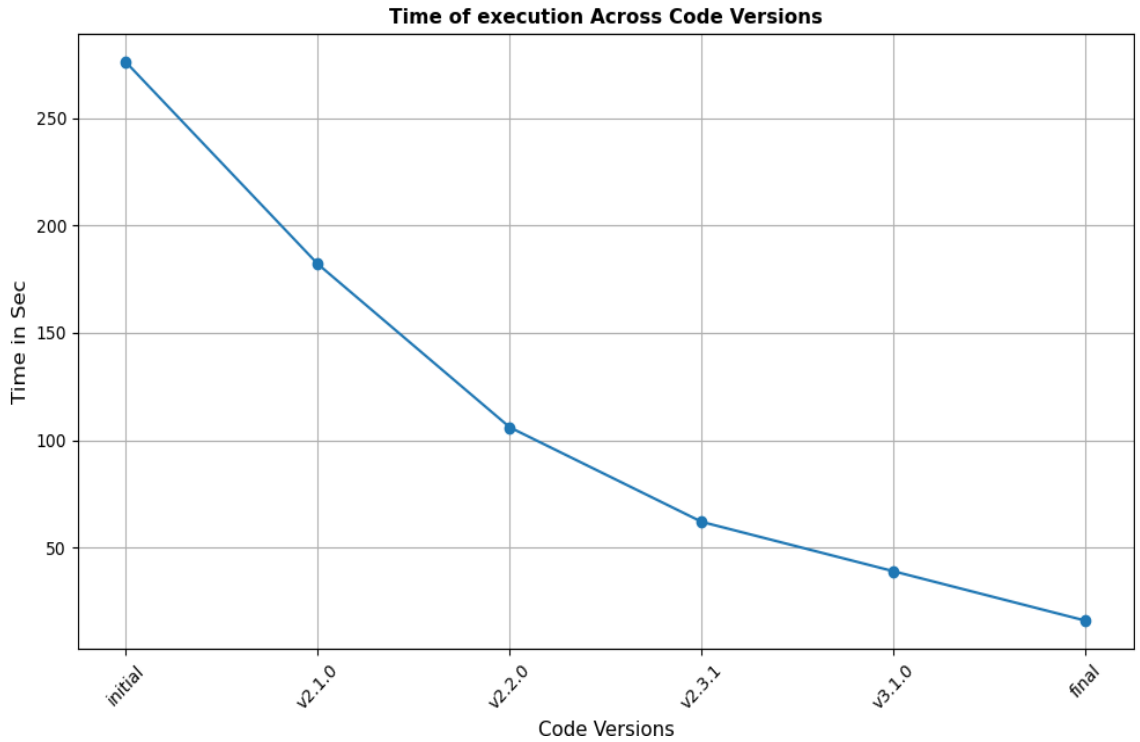
Final Optimized Execution Time: 17,056 ms (-93.8% from base).

Implementation Details:
• Parallel Probing: Threads process distinct pages of the probe-side column, storing results into thread-local threadResults (vector<ColumnT>) using specialized threadInserters.
• Work Stealing Mechanism: Implemented a dynamic load balancing system using an atomic counter. Threads request batches of 4 pages inside a while-loop. When a thread completes its batch, it "steals" the next available batch, ensuring no thread remains idle while others are overloaded.
• Result Merging: Local results are merged by a single thread at the end of the phase to avoid synchronization overhead during execution.

• Work Stealing Mechanism: Implemented a dynamic load balancing system using an atomic counter. Threads request batches of 4 pages inside a while-loop. When a thread completes its batch, it "steals" the next available batch, ensuring no thread remains idle while others are overloaded.
• Result Merging: Local results are merged by a single thread at the end of the phase to avoid synchronization overhead during execution.

Final Results & Analysis

Execution Time Progression Overview

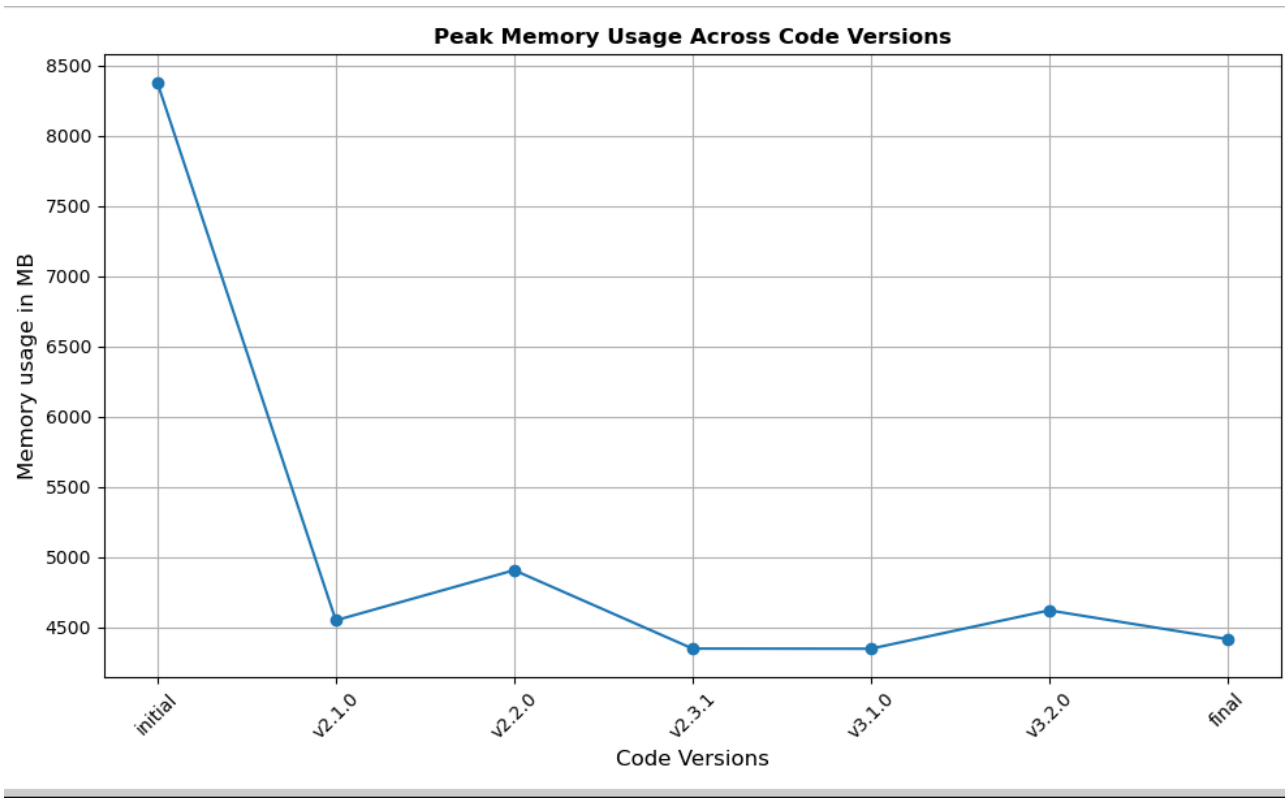


The following table summarizes the average execution times for all major implementations and optimizations throughout the project.

Implementation / Version	Avg Execution Time (ms)	Reduction (vs Base)
Baseline Solution	276,637 ms	0%
Robin Hood Hashing (Phase 1)	271,414 ms	-1.9%
Cuckoo Hashing (Phase 1)	289,445 ms	+4.6% (Slower)
Hopscotch Hashing (Phase 1)	309,102 ms	+11.7% (Slower)
Late Materialization (Phase 2)	182,186 ms	-34%
Column-Store Structure	106,616 ms	-61%
Unchained Hash Table	62,034 ms	-77.5%
Indexing Optimization (Phase 3)	39,256 ms	-85%
Probe Phase Parallelization	19,496 ms	-92%
Work Stealing	17,120 ms	-93.8%
Final (Slab Allocator)	17,056 ms	-93.8%

Memory Consumption

Peak memory usage was halved from 8400 MB to 4500 MB due to Late Materialization. Parallel execution introduced minimal memory overhead.



Scaling Analysis (Threads vs Partitions)

Bits / Partitions	1 Thread	2 Threads	4 Threads	8 Threads
2 bits (4 Parts)	39,438 ms	26,423 ms	18,041 ms	18,178 ms
3 bits (8 Parts)	40,244 ms	26,261 ms	17,917 ms	18,182 ms
4 bits (16 Parts)	40,900 ms	25,610 ms	17,990 ms	18,156 ms
5 bits (32 Parts)	40,839 ms	25,758 ms	18,111 ms	18,123 ms

Optimal performance was observed with 8 partitions on 4 threads. Increasing beyond 4 threads led to context-switching overhead as the hardware only features 4 physical cores.